

```
# Importing the Necessary Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# Importing our dataset
df = pd.read_csv("titanic.csv")
df
```

| PassengerId | Survived | Pclass | Name | Sex                                               | Age    | SibSp | Parch | Ticket | Fare             | Cabin   | Embarked |     |                                                                                     |
|-------------|----------|--------|------|---------------------------------------------------|--------|-------|-------|--------|------------------|---------|----------|-----|-------------------------------------------------------------------------------------|
| 0           | 1        | 0      | 3    | Braund, Mr. Owen Harris                           | male   | 22.0  | 1     | 0      | A/5 21171        | 7.2500  | NaN      | S   |  |
| 1           | 2        | 1      | 1    | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0  | 1     | 0      | PC 17599         | 71.2833 | C85      | C   |                                                                                     |
| 2           | 3        | 1      | 3    | Heikkinen, Miss. Laina                            | female | 26.0  | 0     | 0      | STON/O2. 3101282 | 7.9250  | NaN      | S   |                                                                                     |
| 3           | 4        | 1      | 1    | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0  | 1     | 0      | 113803           | 53.1000 | C123     | S   |                                                                                     |
| 4           | 5        | 0      | 3    | Allen, Mr. William Henry                          | male   | 35.0  | 0     | 0      | 373450           | 8.0500  | NaN      | S   |                                                                                     |
| ...         | ...      | ...    | ...  | ...                                               | ...    | ...   | ...   | ...    | ...              | ...     | ...      | ... |                                                                                     |
| 886         | 887      | 0      | 2    | Montvila, Rev. Juozas                             | male   | 27.0  | 0     | 0      | 211536           | 13.0000 | NaN      | S   |                                                                                     |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
inputs = df[['Pclass', 'Sex', 'Age', 'Fare']]
target = df['Survived']
```

```
# Changing name of columns
inputs.columns = ['pclass', 'sex', 'age', 'fare']
target.columns = ['survived']
```

```
# Checking if there are null values present in the dataset or not
inputs.isnull().sum()
```

```
0
pclass  0
sex      0
age    177
fare     0

dtype: int64
```

```
# Calculating the mean of the Age column because this column has 177 Null Values
inputs['age'].median()
```

28.0

```
inputs['age']
```

|     | age  |
|-----|------|
| 0   | 22.0 |
| 1   | 38.0 |
| 2   | 26.0 |
| 3   | 35.0 |
| 4   | 35.0 |
| ... | ...  |
| 886 | 27.0 |
| 887 | 19.0 |
| 888 | NaN  |
| 889 | 26.0 |
| 890 | 32.0 |

891 rows × 1 columns

dtype: float64

```
# Substituting the Null values in the column with the mean of the Age column
inputs['age'] = inputs['age'].fillna(inputs['age'].median())
```

/tmp/ipykernel\_7689/3173670243.py:1: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row\_indexer,col\_indexer] = value instead

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning)  
inputs['age'] = inputs['age'].fillna(inputs['age'].median())

```
inputs.isnull().sum()
```

|        | 0 |
|--------|---|
| pclass | 0 |
| sex    | 0 |
| age    | 0 |
| fare   | 0 |

dtype: int64

```
inputs.dtypes
```

|        | 0       |
|--------|---------|
| pclass | int64   |
| sex    | object  |
| age    | float64 |
| fare   | float64 |

dtype: object

```
inputs['age']
```

```

      age
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
...     ...
886   27.0
887   19.0
888   28.0
889   26.0
890   32.0

```

891 rows × 1 columns

dtype: float64

```
# Changing the Sex column into numbers
from sklearn.preprocessing import LabelEncoder
le_sex = LabelEncoder()
```

```
inputs['sex_n'] = le_sex.fit_transform(inputs['sex'])
```

/tmp/ipykernel\_7689/3692064025.py:1: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row\_indexer,col\_indexer] = value instead

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning)  
inputs['sex\_n'] = le\_sex.fit\_transform(inputs['sex'])

inputs

|     | pclass | sex    | age  | fare    | sex_n |                                                                                     |
|-----|--------|--------|------|---------|-------|-------------------------------------------------------------------------------------|
| 0   | 3      | male   | 22.0 | 7.2500  | 1     |  |
| 1   | 1      | female | 38.0 | 71.2833 | 0     |  |
| 2   | 3      | female | 26.0 | 7.9250  | 0     |                                                                                     |
| 3   | 1      | female | 35.0 | 53.1000 | 0     |                                                                                     |
| 4   | 3      | male   | 35.0 | 8.0500  | 1     |                                                                                     |
| ... | ...    | ...    | ...  | ...     | ...   |                                                                                     |
| 886 | 2      | male   | 27.0 | 13.0000 | 1     |                                                                                     |
| 887 | 1      | female | 19.0 | 30.0000 | 0     |                                                                                     |
| 888 | 3      | female | 28.0 | 23.4500 | 0     |                                                                                     |
| 889 | 1      | male   | 26.0 | 30.0000 | 1     |                                                                                     |
| 890 | 3      | male   | 32.0 | 7.7500  | 1     |                                                                                     |

891 rows × 5 columns

Next steps: [Generate code with inputs](#) [New interactive sheet](#)

```
# Dropping the sex column and retaining the sex_n column which has numeric values
inputs_n = inputs.drop(['sex'], axis='columns')
inputs_n
```

|     | pclass | age  | fare    | sex_n |                                                                                                                                                                       |
|-----|--------|------|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | 3      | 22.0 | 7.2500  | 1     | <br> |
| 1   | 1      | 38.0 | 71.2833 | 0     |                                                                                                                                                                       |
| 2   | 3      | 26.0 | 7.9250  | 0     |                                                                                                                                                                       |
| 3   | 1      | 35.0 | 53.1000 | 0     |                                                                                                                                                                       |
| 4   | 3      | 35.0 | 8.0500  | 1     |                                                                                                                                                                       |
| ... | ...    | ...  | ...     | ...   |                                                                                                                                                                       |
| 886 | 2      | 27.0 | 13.0000 | 1     |                                                                                                                                                                       |
| 887 | 1      | 19.0 | 30.0000 | 0     |                                                                                                                                                                       |
| 888 | 3      | 28.0 | 23.4500 | 0     |                                                                                                                                                                       |
| 889 | 1      | 26.0 | 30.0000 | 1     |                                                                                                                                                                       |
| 890 | 3      | 32.0 | 7.7500  | 1     |                                                                                                                                                                       |

891 rows × 4 columns

Next steps: [Generate code with inputs\\_n](#) [New interactive sheet](#)

```
x = inputs_n
y = target
```

```
# Doing training and testing of the model
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size = 0.3, random_state = 42)
```

```
# Setting up the parameters
parameters = {
    'criterion' : ['gini', 'entropy', 'log_loss'],
    'splitter' : ['best', 'random'],
    'max_depth' : [3,4,5,6],
    'max_features' : ['sqrt', 'log2'],
    'min_samples_leaf' : [2,5,10,20]
}
```

```
from sklearn.model_selection import GridSearchCV
```

```
# Fitting the model with the parameters with the help of GridSearchCV (Pre-Pruning)
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
cv = GridSearchCV(model, param_grid = parameters, cv = 5, scoring = 'accuracy')
cv.fit(x_train, y_train)
```

GridSearchCV

best\_estimator\_:

DecisionTreeClassifier

DecisionTreeClassifier

```
# Checking the Rsquared of the training dataset
cv.score(x_train,y_train)
```

0.8089887640449438

```
# Retaining the best parameters
cv.best_params_
```

```
{'criterion': 'entropy',
 'max_depth': 3,
 'max_features': 'log2',
```

```
'min_samples_leaf': 10,  
'splitter': 'best'}
```

```
y_pred = cv.predict(x_test)  
y_pred
```

```
array([0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1,  
       0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,  
       1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,  
       0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0,  
       0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0,  
       1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0,  
       0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0,  
       0, 0, 0, 0])
```

```
# Checking R-squared of the test dataset  
from sklearn.metrics import accuracy_score, classification_report
```

```
score = accuracy_score(y_pred, y_test)  
score
```

```
0.7835820895522388
```

```
score
```

```
0.7835820895522388
```

```
# Printing Classification Report  
print(classification_report(y_pred, y_test))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.97      | 0.74   | 0.84     | 205     |
| 1            | 0.52      | 0.92   | 0.67     | 63      |
| accuracy     |           |        | 0.78     | 268     |
| macro avg    | 0.75      | 0.83   | 0.75     | 268     |
| weighted avg | 0.86      | 0.78   | 0.80     | 268     |

```
# Plotting the Decision Tree of the dataset  
from sklearn import tree  
plt.figure(figsize = (20,20))  
tree.plot_tree(cv.best_estimator_ ,filled = True)  
plt.show()
```

